



Document Prepared By **Hamdan Ali**



NAME : HAMDAN ALI

SOFTWARE ENGINEERING INTERNSHIP

Week 8 — Day 38

Authentication, CRUD And the API Layer — Actually Running It

Every command I ran to get the Day 38 code live inside a real Laravel project

Project: TaskFlow — Laravel Project And Task Management System

Submitted by: Hamdan Ali

BS Software Engineering — Air University, Islamabad

Environment: XAMPP (PHP 8.2), Composer, VS Code, PowerShell — Windows

1. What I actually did on this day

Day 38 in my planning folder was already written — Policies, Form Requests, API Resources, the Auth controllers, the full API controller set. But writing the code and having it actually run inside a real Laravel project turned out to be two very different things. This document is my honest record of what I typed into PowerShell, in the order I typed it, to take the Day 38 code from "sitting in a folder" to "actually working inside my taskflow project that's already running Day 36 and Day 37."

I'm writing this the way it actually happened, including the mistake I made at the start, because I think that's more useful for me to look back on later than pretending it went perfectly the first time.

2. Getting back into the right folder

The very first thing I did was mess up my working directory. I had my terminal sitting inside Week8\Day38 (the planning/reference folder) instead of inside my actual Laravel project folder called taskflow, so my very first Copy-Item command failed instantly because PowerShell was trying to find a path that didn't exist. Lesson for myself: always check where I actually am before running a copy command, especially since my project folder and my reference folder both have a "Day38" name floating around them.

```
cd "D:\internship ( HindukushSoft Technologies )\taskflow"  
dir
```

Once dir showed me app, routes, artisan, composer.json etc., I knew I was finally in the right place to start copying things in.

3. Creating the folders Laravel doesn't ship by default

This one surprised me a bit. I assumed app\Http\Requests, app\Http\Resources, and app\Policies would already exist in a fresh Laravel install, but they don't — Laravel only creates them the first time you actually generate something with artisan, or you make them yourself. Since I was copying files in manually instead of using php artisan make:policy etc., I had to create the destination folders myself first, otherwise Copy-Item just says "directory name is invalid" and refuses to copy anything.

```
New-Item -ItemType Directory -Force -Path "app\Http\Requests"  
New-Item -ItemType Directory -Force -Path "app\Http\Resources"  
New-Item -ItemType Directory -Force -Path "app\Policies"  
New-Item -ItemType Directory -Force -Path "app\Http\Controllers\Api"
```

4. Copying in all four groups of Day 38 files

With the folders in place, the actual copy was simple — one command per group, straight from my Week8\Day38 planning folder into the matching spot inside taskflow.

```
Copy-Item "..\Week8\Day38\app\Http\Requests\*.php" "app\Http\Requests\" -Force
Copy-Item "..\Week8\Day38\app\Http\Resources\*.php" "app\Http\Resources\" -Force
Copy-Item "..\Week8\Day38\app\Policies\*.php" "app\Policies\" -Force
Copy-Item "..\Week8\Day38\app\Http\Controllers\Api\*.php" "app\Http\Controllers\Api\" -Force
```

Then I checked every single folder afterwards instead of just assuming it worked, since I'd already been burned once this project by a copy silently going somewhere weird:

```
dir app\Http\Requests
dir app\Http\Resources
dir app\Policies
dir app\Http\Controllers\Api
```

All four folders came back with the right file counts — 5 Form Requests, 4 API Resources, 3 Policies, and 6 API controllers. That matched what I expected from my own planning docs, so I moved on.

5. Setting up routes/api.php

This part actually turned out to be less work than I thought, because it had apparently already been generated earlier in the project (I'm guessing from when I installed Sanctum back on Day 37 to get login working). I checked first instead of assuming:

```
dir routes
php artisan install:api
```

Running install:api told me straight up: "API routes file already exists" — so it didn't touch anything, just asked if I wanted to run pending migrations, which I said yes to (it said "Nothing to migrate", since it was already up to date). Then I double checked the actual contents of the file to make sure it genuinely matched my planned route list rather than being some half-finished version:

```
type routes\api.php
```

It matched exactly — all the auth routes, the apiResource for projects, the nested task/comment routes, all wrapped in the auth:sanctum middleware group. Nothing to fix here, which was a nice change of pace.

6. Making sure the User model was actually wired for Sanctum

Since the whole API depends on token auth, I wanted to be completely sure the User model had the HasApiTokens trait properly imported and used, not just assumed it because I remembered writing it that way originally:

```
type app\Models\User.php
```

Confirmed — the import and the trait were both there. Small thing, but worth actually checking instead of trusting my memory of what I wrote days earlier.

7. Verifying the routes and the policies really work

This is the part I actually found most satisfying about Day 38 — not just copying files, but proving Laravel was really recognising them. First, the full route list:

```
php artisan route:list --path=api
```

This printed all 21 routes I expected, every controller method matched what I'd written, nothing missing. Then the real test — whether Laravel's policy auto-discovery actually picked up my three Policy classes just from them sitting in app/Policies with matching names to my models:

```
php artisan tinker --execute="var_dump(Gate::getPolicyFor(App\Models\Project::class));"
```

This printed `object(App\Policies\ProjectPolicy)#7092 (0) {}` — meaning Laravel found it automatically, no manual registration needed. I did the same check for the other two policies together in one command to save time:

```
php artisan tinker --execute="var_dump(Gate::getPolicyFor(App\Models\Task::class));  
var_dump(Gate::getPolicyFor(App\Models\Comment::class)); echo  
Schema::hasTable('personal_access_tokens') ? 'sanctum table OK' : 'MISSING';"
```

All three policies came back correctly wired, and the Sanctum token table was confirmed to exist. At that point I was confident Day 38 was genuinely done — not "the files exist" done, but "Laravel itself is enforcing these rules" done.

8. What I'd tell myself if I did this again

- Check your working directory before running any Copy-Item command — a wrong prompt path wastes more time than it should.
- Laravel 12's default scaffold is leaner than I expected — folders like Requests/Resources/Policies don't exist until you make them, even though tutorials online often assume they're already there.



- Don't assume a copy worked — always dir the destination right after, especially on a project where I'd already had a couple of file-content mixups earlier in the week.
- Policy auto-discovery in Laravel is genuinely convenient once you trust it, but I'm glad I verified it with `Gate::getPolicyFor()` instead of just assuming naming convention would work — it's the kind of thing that's easy to get wrong silently.

9. Result

By the end of this session: all Day 38 Requests, Resources, Policies, and API controllers were copied into the real project, `routes/api.php` matched my planned route map exactly, all 21 API routes were confirmed live via `route:list`, and all three Policies were confirmed auto-wired via `Gate::getPolicyFor()`. Combined with the working Day 36 database and Day 37 UI, this meant the project now had a fully authenticated, policy-protected REST API sitting underneath the same browser-facing app — genuinely two working access paths into the same data.

THANK YOU!